

KAT: Dependency-aware Automated API Testing with Large Language Models

Tri Le
Katalon Inc.

Ho Chi Minh City, Vietnam
tri.qle@katalon.com

Thien Tran
Katalon Inc.

University of Science
Vietnam National University
Ho Chi Minh City, Vietnam
thien.tran@katalon.com

Duy Cao
Katalon Inc.

University of Science
Vietnam National University
Ho Chi Minh City, Vietnam
duy.cao@katalon.com

Vy Le
Katalon Inc.

Ho Chi Minh City, Vietnam
vy.le@katalon.com

Tien N. Nguyen
Computer Science Department
University of Texas at Dallas
Dallas, Texas, USA
tien.n.nguyen@utdallas.edu

Vu Nguyen*
Katalon Inc.
University of Science
Vietnam National University
Ho Chi Minh City, Vietnam
nvu@fit.hcmus.edu.vn

Abstract—API testing has increasing demands for software companies. Prior API testing tools were aware of certain types of dependencies that needed to be concise between operations and parameters. However, their approaches, which are mostly done manually or using heuristic-based algorithms, have limitations due to the complexity of these dependencies. In this paper, we present KAT (Katalon API Testing), a novel AI-driven approach that leverages the large language model GPT in conjunction with advanced prompting techniques to autonomously generate test cases to validate RESTful APIs. Our comprehensive strategy encompasses various processes to construct an operation dependency graph from an OpenAPI specification and to generate test scripts, constraint validation scripts, test cases, and test data. Our evaluation of KAT using **12 real-world RESTful services** shows that it can improve test coverage, detect more **undocumented** status codes, and **reduce false positives** in these services in comparison with a state-of-the-art automated test generation tool. These results indicate the effectiveness of using the large language model for generating test scripts and data for API testing.

Index Terms—REST API, Black-box testing, API testing, Large language models for testing

I. INTRODUCTION

RESTful APIs (or REST APIs - REpresentational State Transfer) is a software architectural style to guide the development of web APIs. RESTful APIs communicate through HTTP requests to perform standard database functions such as creating, reading, updating, and deleting records (CRUD). RESTful API has become one of the most used software architectural styles due to its high flexibility and scalability, as well as being relatively secure and easy to implement.

Automated test cases and data generation for API testing have been an active research topic that has attracted many studies in recent years [1], [2], [3], [4], [5], [6]. These studies can be divided into black-box and white-box test generation approaches. Black-box testing, which is the most common,

uses the OpenAPI Specification (OAS) as a basis to generate test cases and data. White-box testing focuses on analyzing source code to drive test cases and data [3].

While current approaches have made significant progress in testing RESTful APIs, they still face challenges in addressing *intricate dependencies among API endpoints and their parameters*. These dependencies fall into three categories. Firstly, there are dependencies among API endpoints (operations). For instance, to test an endpoint for charging a credit card in an online flight booking system, one must first invoke the operation for selecting the flight. Secondly, there are dependencies among the parameters of an operation. For example, in a flight booking system, the operation requires parameters like `arrivalDate` and `departureDate`. And in that context, the constraint is that `departureDate` must precede `arrivalDate`. Thirdly, there are dependencies between an operation and its parameters. For instance, in a flight booking system, the parameters `arrivalDate` and `departureDate` of a reservation operation must be set in the future.

Unfortunately, the foregoing API testing frameworks have limitations in handling such dependencies. In the case of inter-operation dependencies, RestTestGen [1] employs a heuristic approach centered on name matching to identify relationships among operations. This hinges on a shared field between the output of one operation and the input of another. However, discrepancies in field names could lead the heuristic algorithm to incorrectly establish dependencies. For instance, exclusive reliance on field names might fail to accurately identify the `GET /flights` endpoint as a dependent operation of the `POST /booking` endpoint, since the algorithm might not find matching pairs of field names.

In terms of inter-parameter dependencies, previous methods have indeed considered them in test case generation. Yet, it is crucial to note certain limitations in these approaches. For instance, RESTest [2] addresses this by mandating the

* Corresponding: Vu Nguyen (nvu@fit.hcmus.edu.vn)

manual inclusion of dependencies among parameters in the testing OAS file under the `x-dependencies` field. This manual intervention demands a significant amount of time from testers. Thus, any improvements in simplifying this process would signify progress in this field.

Regarding dependencies between operations and parameters, cutting-edge approaches employ heuristic-based methods (RestTestGen [1]) or rule-based methods (bBOXRT [7]) to generate both valid and invalid test data. However, they might not fully comprehend these constraints articulated in natural language or might necessitate manual intervention.

Through harnessing the excellent capabilities of GPT [8] in interpreting the natural language content embedded within Swagger files for RESTful APIs, our proposed approach, KAT (KATalon API Testing), leverages a powerful tool for comprehending intricate dependencies. This utilization of GPT empowers us to systematically extract and discern the dependencies that exist among the various API endpoints, indicating relationships that define the functionality of the APIs. Moreover, GPT possesses the capability to analyze the connections among the parameters associated with each operation. This enables us to perceive the underlying details of the API, ultimately leading to more thorough testing. In contrast, state-of-the-art methods, which mainly rely on heuristic approaches, fall short in capturing those dependencies.

Our methodology employs a generative language model and advanced prompting techniques at key stages of the testing process. This includes (1) identifying relationships among schemas and operations, which are used to construct an operation dependency graph (ODG), and (2) detecting dependencies among input parameters as well as inter-dependencies between operations and their respective parameters for both valid and invalid test data generation. To streamline this process, our approach seamlessly integrates the ODG into the test script generation. The resulting test scripts are then combined with the generated test data to create a comprehensive suite of test cases, which are subsequently executed on the target APIs.

We have conducted an experiment to evaluate our approach, KAT, using a dataset comprising **12 RESTful API services**. The results demonstrate a significant **improvement of 15.7%** in the coverage of **status codes documented in OAS** files over the state-of-the-art RestTestGen [9]. Additionally, KAT effectively identifies the status codes that are not explicitly specified in the OAS files while reducing the number of false-positive test cases generated.

In this paper, we make the following key contributions:

- 1) KAT, an GPT-based approach to generate tests for RESTful API testing that considers the *inter-dependencies among operations, inter-parameter dependencies, and the dependencies between operations and parameters*.

- 2) An extensive evaluation showing our approach outperforming the state-of-the-art RESTful API testing approaches.

II. MOTIVATING EXAMPLE

A. Observations

In this section, we use an example to elucidate the issue and inspire our approach. Fig. 1 shows a representative OAS file delineating the RESTful APIs for an online flight booking system. Within this OAS file, two endpoints are documented: `GET /flights` and `POST /booking`. The former endpoint serves the purpose of retrieving a list of available flights within a specified date range, while the latter facilitates passengers in reserving a new flight. For each endpoint (operation), the file contains a descriptive summary (e.g., lines 7 and 20), the list of parameters (lines 21–24) and the request body object (line 25) with their names and textual descriptions in the schema (e.g., lines 25–41). Moreover, there exist supplementary endpoints supporting pertinent operations (omitted here for brevity).

From this example, we make the following observations:

Observation 1 (Dependencies among Operations): *The usage of operations involves the dependencies among them.*

A test case of an endpoint might only be able to be successfully tested if the prerequisite test cases of other endpoints have already been properly executed beforehand. This is because responses to these prerequisite endpoints may affect the request for a current endpoint under test.

In the provided OAS (refer to Fig. 1), it is crucial to supply a proper parameter for accurate testing of the `POST /booking` endpoint. This parameter should consist of a single field named `flightId`, representing an existing flight in the database. To meet this condition, it is necessary to first successfully execute the `GET /flights` endpoint. This action will result in a response containing a list of available flights. From this list, a specific flight element can be chosen, and its corresponding `flightId` value extracted. This value is then integrated into the test case for the `POST /booking` operation.

Observation 2 (Dependencies among Parameters): *The usage of an operation in a library involves the dependencies among its parameters.*

Dependencies also exist among the parameters of an operation. Inter-parameter dependency refers to a constraint that exists between two or more input parameters of an API endpoint. This constraint may pertain to the required values of these parameters, which must satisfy a predefined condition. Alternatively, it could involve the presence of an optional parameter, rendering the absence of another parameter problematic. For instance, consider the endpoint `POST /booking`. Here, a tangible inter-parameter constraint is evident between `departureDate` (line 30) and `arrivalDate` (line 34), an association discernible to users. Naturally, it is expected that the departure date of a flight precedes its arrival date. Thus, when handling the result, the requesting parameters should account for this constraint to ensure a valid response.

Observation 3 (Dependencies between An Operation and Its Parameters): *The usage of an operation involves the dependencies between that operation and its parameters.*

Certain endpoints may encompass fields with values that must adhere to constraints, requiring those values to make

```

1 openapi: 3.0.0
2 info:
3   title: Flight Booking API
4 paths:
5   /flights:
6     get:
7       summary: Get Flights
8       responses:
9         '200':
10          description: A list of available flights.
11          content:
12            application/json:
13              schema:
14                type: array
15                items:
16                  $ref: '#/components/schemas/Flight'
17
18   /booking:
19     post:
20       summary: Book a new Flight
21       parameters:
22         - name: flightId
23           schema:
24             type: integer
25       requestBody:
26         content:
27           application/json:
28             schema:
29               properties:
30                 departureDate:
31                   type: string
32                   format: date
33                   description: format in YYYY-MM-DD.
34                     Should be after today.
35                 arrivalDate:
36                   type: string
37                   format: date
38                   description: format in YYYY-MM-DD.
39                     Should be after 'departureDate'.
40                 passengerName:
41                   type: string
42                 passengerAge:
43                   type: integer
44       responses:
45         '200':
46          description: The booking is successful.
47          content:
48            application/json:
49              schema:
50                $ref: '#/components/schemas/Booking'
51
52 components:
53   schemas:
54     Flight:
55       type: object
56       properties:
57         id:
58           type: integer
59         origin:
60           type: string
61         destination:
62           type: string
63     Booking:
64       type: object
65       properties:
66         flight:
67           $ref: '#/components/schemas/Flight'
68         departureDate:
69           type: string
70           format: date
71         arrivalDate:
72           type: string
73           format: date
74         passengerName:
75           type: string
76         passengerAge:
77           type: integer

```

Fig. 1. An Example of OpenAPI/Swagger Specification (OAS) file

sense in a real-world context. For instance, within the OAS file in Fig. 1, the value of `passengerAge` specified in the `POST /booking` endpoint (line 40) must exceed zero, although this constraint is not explicitly expounded within the OAS file. Furthermore, the value assigned to `departureDate` must refer to a future date. Unlike the `passengerAge` parameter, this constraint is articulated in natural-language description.

B. The state-of-the-art Approaches

Despite that the dependencies exist among the endpoints and parameters as observed, the state-of-the-art API testing approaches do not sufficiently address and capture them.

Regarding the inter-operation dependencies, RestTestGen employs a *heuristic approach based on name matching*. This determination hinges on the presence of a shared field between the output of one operation (i.e., endpoint) and the input of another. For instance, in the OAS file in Fig. 1, the `GET /flights` endpoint yields a set of Flight objects, each featuring an `id` field. Conversely, the `POST /booking` endpoint anticipates a `flightId` field as part of its input. This disparity in field nomenclature implies that the heuristic algorithm might encounter difficulty in establishing dependencies or might potentially mis-attribute them to other endpoints sharing similar field names. Relying solely on field names, it may not correctly identify the `GET /flights` endpoint as a dependent operation of the `POST /booking` endpoint.

When it comes to inter-parameter dependencies, previous approaches have indeed taken them into account during the test case generation. However, it is important to acknowledge certain limitations in these methods. For example, RESTest [2] addresses this issue by requiring *manual inclusion of dependencies among parameters* in the testing OAS file, utilizing the `x-dependencies` field. This manual intervention demands a considerable amount of time from testers. Regarding the dependencies between the operations and parameters, the state-of-the-art approaches use heuristic-based (RestTestGen) or rule-based (bBOXRT) methods to generate valid and invalid test data for testing, but they may not fully address these constraints described in natural language or require manual intervention to specify the dependencies.

C. Capability of GPT and Motivation

The Large Language Models (LLMs) have been achieving remarkable successes in understanding texts and source code [8]. In this work, to capture such dependencies, we propose to explore that capability of the LLMs. To motivate this point, we made a few prompts to GPT-3.5 [8] to explore its capability in understanding the intricate dependencies among the API endpoints (or operations) and their parameters. In particular, we use the version of `gpt-3.5-turbo-1106` via the provided API from OpenAI to request. We first loaded the Swagger file in Fig. 1 and constructed prompts. The following conversations with GPT show how it can address the dependency problem:

Prompt: “Are there any dependencies between the `POST /booking` operation and `GET /flights` operation?”
GPT’s response: “Yes, there is a dependency between the `POST /booking` operation and the `GET /flights` operation. Before making a booking, the user needs to retrieve the available flights by calling the `GET /flights` operation. This is necessary to obtain `flightId`, which is required in the request body of the `POST /booking` operation. Thus, the `GET /flights` operation is a prerequisite operation that is called before the `POST /booking` operation can be tested successfully.”

GPT is also able to detect inter-parameter dependencies:

Prompt: “Are there any inter-parameter dependencies mentioned in the POST /booking operation spec?”

GPT’s response: “Yes, there are inter-parameter dependencies mentioned in the POST /booking operation spec. The arrivalDate parameter is dependent on the departureDate parameter. The description states that the arrivalDate should be after the departureDate. Therefore, the departureDate needs to be provided before the arrivalDate in order to ensure that the request is valid.”

Finally, GPT is capable of understanding the constraints among the parameters of the operations:

Prompt: “Are there any constraints of the value of parameters appeared in the specification, which are not defined in the specification but worth noting in a real-world context?”

GPT’s response: “In the provided specification, there are no specific constraints for the parameter values. However, in a real-world context, the following constraints are worth noting:

1. departureDate: Must be a future date (YYYY-MM-DD) and cannot be in the past.
 2. arrivalDate: Must be after the departureDate (YYYY-MM-DD) and cannot be the same as or before the departureDate.
 3. passengerAge: Subject to minimum and maximum age requirements, age categories for fare differentiation, or legal age restrictions for specific flights or destinations.
- These constraints ensure the validity of flight and passenger details, aligning with the considerations for flight bookings.”

In brief, GPT is capable of identifying the relationships between operations and their parameters on a token level. Thus, we leverage GPT in producing the test cases that cover those dependencies among the APIs and/or their parameters.

III. DEPENDENCY-AWARE AUTOMATED API TESTING

A. Approach Overview

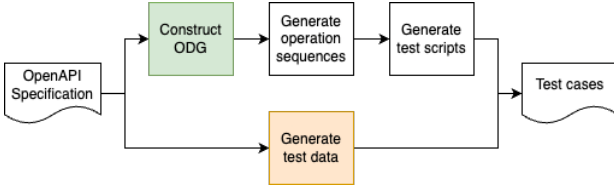


Fig. 2. KAT: Dependency-aware Automated API Testing

This section presents an overview of our approach, KAT (Fig. 2) that leverages LLMs, GPT in particular, to construct ODGs and generate test scripts and data for API testing. KAT receives the OAS file as input. This file serves as the basis for extracting the detailed information about its service to construct ODG and generate test scripts and data. KAT consists of the following steps:

- **Construct ODG:** this step aims to construct an ODG representing dependencies between operations defined in the specification. See more details in Section III-B.
- **Generate operation sequences:** using the ODG obtained from the previous step, our approach generates sequences of operation executions or requests in which one request prepares the data (parameters and request body) needed for succeeding requests. Dependent on specific ODG, it is possible that an operation is standalone or independent,

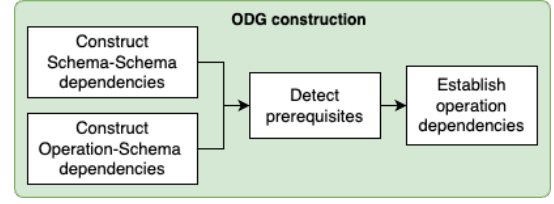


Fig. 3. ODG Construction

which does not require any preceding requests to prepare the data needed for its successful execution.

- **Generate test scripts:** this step aims to generate test scripts using the obtained operation sequences.
- **Generate test data:** details of this step, which generates the data for a test operation, are given in Section III-C.

B. Operation Dependency Graph Construction (Fig. 3)

1) Important Concepts:

Definition 1 (Operation Dependency Graph): An ODG is represented as a directed graph $G = (N, V)$, wherein each node N signifies a discrete operation within the RESTful API. The presence of an edge $v \in V$, articulated as $v = n_1 \rightarrow n_2$, signifies the existence of a dependency between the nodes n_1 and n_2 . This dependency is predicated on the condition that one or more fields in the output of source node n_1 must coincide with in the input of target node n_2 , thereby mandating that the execution of operation n_1 precedes that of n_2 .

The concept of ODG was previously explored in the work of RestTestGen. Refer to the exemplar OAS file in Fig. 1, and observe a dependency between the two endpoints, GET /flights and POST /booking, as explained in Observation 1 (Fig. 1). Part of the ODG of this OAS file is shown in Fig. 4.

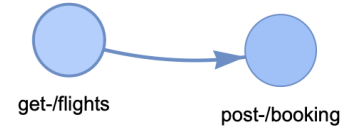


Fig. 4. Dependence between GET/flights and POST/booking in ODG

2) **ODG Construction Algorithm:** Our algorithm differs from the heuristic-based ODG construction algorithm in RestTestGen [1] in its utilization of GPT. Algorithm 1 presents our pseudocode, which involves the use of GPT for inferring the dependencies between operations and schemas, as well as the dependencies among schemas. These process steps are employed to construct additional operation dependency edges that heuristic-based algorithms cannot detect.

Heuristic-based collection of edges: Primarily, the function `gatherODheuristic` (line 2) is employed to iterate through all operations, attempting to match input-output pairs. Upon detecting a pair with a perfect character match, we add the resulting edge between the corresponding nodes in the graph.

In the case of the OAS file depicted in Fig. 1, the relationship between two operations is considered. Although the pair

Algorithm 1: ODG Construction

```

1 Function generateOperationDependencies (Swagger)
   Input : A Swagger Specification file
   Output: List of Operation Dependencies OD
2   OD  $\leftarrow$  gatherODheuristic (Swagger)
3   OS  $\leftarrow$  GPTgenOperationSchemaDep (Swagger)
4   SS  $\leftarrow$  GPTgenSchemaSchemaDep (Swagger)
5   O  $\leftarrow$  getAllOperationsFrom (Swagger)
6   foreach Oi in O do
7     P  $\leftarrow$  getParameters (Oi)
8     PSDep  $\leftarrow$  getDep (OS, Oi)
9     if PSDep  $\neq \emptyset$  then
10      NewOD  $\leftarrow$  gatherOperationDep (PSDep, P)
11      OD.extend(NewOD)
12    end
13    if P  $\neq \emptyset$  then
14      ChildS  $\leftarrow$  getDep (SS, Oi)
15      NewOD  $\leftarrow$  gatherOperationDep (ChildS, P)
16      OD.extend(NewOD)
17    end
18  end
19  return OD
20 Function gatherOperationDep (C, P)
   Input : A collection to find new OD elements C
           A set of parameters to check dependency P
   Output: A set of new elements to be extended NewOD
21  foreach Cj in C do
22    PO  $\leftarrow$  findPrecedingOperations (Cj)
23    if PO  $\neq \emptyset \vee PO \in P$  then
24      extendDependencies (NewOD, PO)
25    end
26    P.remove(PO)
27  end
28  return NewOD

```

flightId (line 22, input of POST /booking) and id (existed in the schema “Flight”, the output of GET /flights, on line 16) do not exhibit a perfect match, a significant dependency persists due to the presence of id within the “Flight” schema, semantically aligning with flightId. To resolve this, we use GPT to analyze such pairs in a two-step process, involving the establishment of Operation-Schema dependencies (line 3) and Schema-Schema dependencies (line 4), thereby acting as a bridge to identify the dependencies between operations.

Operation-Schema Dependency:

Definition 2 (Operation-Schema Dependency): The origin of two operations depends on each other when the response of one operation is required as input for the successful execution of the other. Within an OAS, each operation’s response is defined under a schema, and subsequent operations must identify relevant schemas to determine their predecessor operations. An operation-schema dependency encompasses all schemas that share keys between their fields and the input parameters of the operation, which GPT can effectively identify.

Operation-Schema dependencies capture this relationship by constructing the dictionary *OS* (line 3). In this dictionary, each key corresponds to an operation’s name $o_i \in O_n$, in which O_n is a set of operation’s name described in the OAS specification. This key is associated with a collection of data, providing insights into essential prerequisite schemas and the connections between the current operation o_i and these schemas through their parameters. For each parameter pair, the first property

```

1 OS = {
2   "post-/booking": {
3     "Flight": {
4       "flightId": "id"
5     }
6     "Booking": {
7       "flightId": "flight"
8     }
9   }
10 }

```

Fig. 5. Example of an Operation-Schema dependency dictionary. The operation POST /booking has two Operation-Schema dependencies with the schemas “Flight” and “Booking”. This is indicated by the pairs of parameter flightId (line 22) with field id (line 55) and flight (line 64) in Fig 1.

```

1 PROMPT="""Given the operation and its parameters,
2   identify all prerequisite
3   schemas for retrieving information related to the
4   operation’s parameters.
5
6 Below is the operation and its parameters:
7 post-/booking:
8   flightId: integer
9   departureDate: string
10  arrivalDate: string
11  ...
12
13 Below is the list of all schemas and their properties:
14 schemas:
15   Flight:
16     ...
17   Booking:
18     ...
19
20 Please format the prerequisite schemas in the following
21 structure:
22 <parameter of the operation> -> <equivalent operation of
23   the relevant schema>
24 ...""

```

Fig. 6. The prompt for prerequisites in the POST /booking operation yields a GPT response with the “Flight” and “Booking” schemas, creating Operation-Schema dependencies.

belongs to the schema, while the second property belongs to the operation o_i . As an example, consider the *OS* dictionary created for the OAS file shown in Fig. 1, which is illustrated in Fig. 5. Fig. 6 shows the prompt we used to infer all prerequisite schemas of the operation POST /booking.

Fig. 5 shows the operation POST /booking along with two schemas, “Flight” and “Booking,” identified as its prerequisite schemas by GPT. When detecting the “Flight” schema, it allows to infer that GET /flights is the predecessor since the “Flight” schema is specified as the reply for GET /flights. However, in some cases, GPT only identifies the “Booking” schema, and cannot identify a clear predecessor operation since there is no operation specifying “Booking” as its response. To address this, we use a Schema-Schema dependency.

Schema-Schema Dependency:

Definition 3 (Schema-Schema Dependency): We define S_n is a set of n schemas described in the OAS specification. Schema-schema dependencies model the relationship between schemas by initializing the dictionary *SS* (line 4). This dictionary contains n distinct key-value pairs where key is a schema $s_i \in S_n$, and value is a list of schemas, where:

- Each schema in the list contains at least one field that refers to a field specified in the key, or
- The schema itself is referred by the key (e.g. the element “Flight” in the value is referred by the key “Booking”).

```

1 PROMPT="""Given the schema and its properties in the
  OpenAPI specification of an API application, your
  task is to identify the prerequisite schemas that
  need to be created before establishing the mentioned
  schema.
2
3 Below is the schema and its properties
4 Booking:
5   flight:
6     $ref: '#/components/schemas/Flight/'
7   departureDate: ...
8   arrivalDate: ...
9   ...
10
11 Below is the list of all schemas and their properties:
12 schemas:
13   Flight:
14     ...
15   Booking:
16     ...
17
18 Return in separated lines. No explanation needed."""

```

Fig. 7. The prompt for “Booking” prerequisites elicits a GPT response with the “Flight” schema, establishing a Schema-Schema dependency.

Consider the conversation displayed in Fig. 7 with GPT. Our approach tries to obtain potential relationships of the schema “Booking” using the Swagger displayed in Fig. 1.

After iterating through all schemas, the *SS* for the OAS file given in Fig. 1 will have the value as shown below.

```

1 SS = {
2   "Flight": [],
3   "Booking": [
4     "Flight"
5   ]
6 }

```

The process to detect prerequisites and construct edges of the graph: Upon the completion of constructing these two hierarchical sets generated by the GPT model, we initiate a process to establish dependencies between operations.

The process has a `for` loop (lines 6–18), iterating through extracted operations from the OAS file (line 5). The objective is to identify the relevant schema for each parameter obtained from the current operation (line 7). Using the Operation-Schema dependency stored in *OS* (line 8), *OD* is updated (line 11). Any unaccounted parameters at the current schema level prompt an attempt to retrieve the next schema level using *SS* (line 14), followed by a second attempt to update the final result (line 16). These steps establish the adequacy of *OD* to generate the sequential order of operations for test generation.

Specifically, the function `gatherOperationDep` (line 20) takes two inputs. It contains a single loop (line 21), iterating through all elements in the collection. Each element is examined by `findPrecedingOperations` (line 22). This function returns a list of preceding operations (*PO*), representing POST or GET operations, indicating the need for creating a new data item before executing it on any test cases or ensuring the existence of that data in the database. After this, *P* checks the edges to determine if *PO* is usable for the current operation.

C. Test Data Generation

This section describes our approach to leverage GPT to generate both valid and invalid test data, as well as Python scripts to validate the accurate correspondence between parameters. We use GPT to generate high-quality valid and invalid test

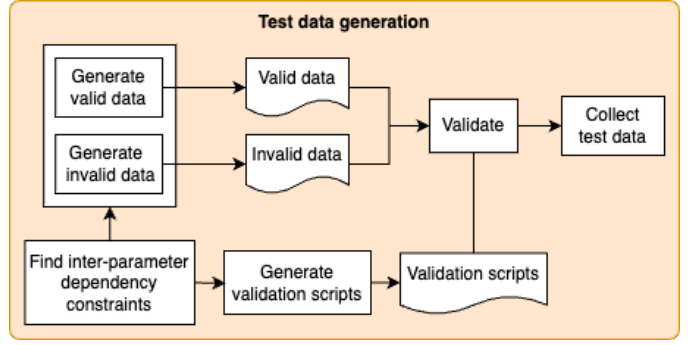


Fig. 8. The process of test data generation for a single operation

```

1 GET_DATASET_PROMPT=f"""Given the information about the
  operation, generate a
2 dataset containing 10 data items to be used to test the
  operation.
3 {additional_instruction}
4
5 Operation information: {endpoint_information}
6 Referenced schema: {ref_schema}
7
8 Your dataset represents each data item in the JSONL
  format, line by line."""

```

Fig. 9. Sample prompt template for test data generation

data that closely mimic real-world data. Additionally, GPT is used to identify inter-dependencies among the operation’s parameters. These dependencies serve as context for dynamically update the data generation, and are used for formulating Python validation scripts. When creating test data files for failure cases, we consider various scenarios, such as omitting values in required fields, providing incorrect data types for certain fields, or violating inter-parameter constraints.

These Python validation scripts ensure consistency with the data file. However, if no inter-parameter dependencies are identified in the current operation, the Python script generation process does not take place, and no additional update in data generation. Following another cycle of data generation, each item in the valid/invalid data files undergoes an additional evaluation phase before being compiled into usable test data files. These files are then integrated with a test script to form a test case for the operation. JSON format is chosen for communication between a data item and its script.

All prompts follow the format shown in Fig. 9. The section labeled `additional_instruction` is employed to guide the model in generating valid or invalid datasets and to specify failed scenarios for more comprehensive coverage of failures.

An illustration for the generated data, which represents the value of the request body for `POST /booking` is displayed in Fig. 1. The valid and invalid data is as shown in Fig. 10.

D. Generating Test Scripts and Test Cases

We adhere to the approach of examining response status codes within the 2xx and 4xx ranges. For an operation, a corresponding test script is generated by the GPT model, based on the number of sequences extrapolated from the ODG. The leading operation in each order retrieves the data element from

```

1 [
2   {
3     "data": {
4       "departureDate": "2022-12-01",
5       "arrivalDate": "2022-12-02",
6       "passengerName": "John Doe",
7       "passengerAge": 30
8     },
9     "expected_code": 200
10  },
11  {
12    "data": {
13      "departureDate": "2022-11-15",
14      "arrivalDate": "2022-11-16",
15      "passengerName": "Jane Smith",
16      "passengerAge": 25
17    },
18    "expected_code": 200
19  },
20  ...
21 ]

```

```

1 [
2   {
3     "data": {
4       "departureDate": "2022-03-10",
5       "arrivalDate": "2022-03-09",
6       "passengerName": "Michael Johnson",
7       "passengerAge": 35
8     },
9     "expected_code": 400
10  },
11  {
12    "data": {
13      "departureDate": null,
14      "arrivalDate": "",
15      "passengerName": "John Doe",
16      "passengerAge": "25"
17    },
18    "expected_code": 400
19  },
20  ...
21 ]

```

Fig. 10. These items illustrate valid and invalid request body data for the POST /booking endpoint.

its relevant data file, and its output is assimilated as the input of the successive operation in the sequential procession. Each operation is furnished with a tailored test script that exclusively draws a data element from the test data file, housing only a singular execution of that operation in and of itself.

Fig. 11 displays a generated Groovy programming language test script for consecutively executing two requests to test the operation POST /booking, as described in Fig. 1. It illustrates how the operation dependencies impact the order of execution of different operations in a single script. Custom keywords like `makeRequest` and `assertStatusCode` have been created to facilitate clean Groovy code, making it easier for the GPT model to generate an executable test script.

After establishing a collection for evaluating 2xx status codes, we used these scripts as templates to create scripts for assessing 4xx status codes. These existing scripts can be adjusted by modifying the path of relevant test data files and rearranging the order within the operation sequence. For example, if a pre-existing DELETE operation aligns with the current element in the sequence, it can be inserted to simulate a scenario of receiving a 404 status code.

IV. EMPIRICAL EVALUATION

A. Research Questions

For evaluation, we seek to answer the following questions:

```

1 // Import statements
2 import ...
3
4 // ChatGPT generated test data goes here
5 def path_variables_1 = [:]
6 def query_parameters_1 = [:]
7 def body_1 = ""
8
9 def response_1 = makeRequest(
10   <path to test GET /flight>, path_variables_1,
11   query_parameters_1, body_1
12 )
13
14 // ChatGPT generated test data goes here
15 def path_variables = [:]
16 def query_parameters = [
17   'flightId': response_1[0].id
18 ]
19 def body = <get from the relevant data file>
20
21 def response = makeRequest(
22   <path to test POST /booking>, path_variables,
23   query_parameters, body
24 )
25 def latest_response = response
26
27 // END
28 def expected_status_code = <value>
29 assertStatusCode(latest_response, expected_status_code)

```

Fig. 11. The Groovy-generated test script exemplifies the POST /booking operation following the GET /flight → POST /booking sequence.

RQ1. [Test coverage]: How well does our approach generate valid test cases for coverage improvement?

RQ2. [Test generation efficiency]: How efficient is our approach in generating valid test cases?

RQ3. [Failure detection]: What is the capability of our approach in detecting failures and mismatches between the working API and its specification?

We chose to compare our approach against the state-of-the-art RestTestGen (RTG for short).

For RQ1, our analysis involves using RTG and our approach (KAT) to generate test cases and run them on the target APIs to measure the test coverage of successful (2xx) and operation failure responses (4xx).

For RQ2, we evaluate the efficiency of the approaches by measuring the efficiency score which is the number of test cases triggering 2xx and 4xx responses over the total number of test cases generated. A higher efficiency score indicates a more efficient approach, which also means fewer test cases needed to generate for covering status codes.

For RQ3, we measure the number of 500 errors found, the number of status codes not documented in the OAS, and the number of mismatches in status codes defined in the OAS and those returned while making requests to target services.

B. Dataset

The dataset used in our experimentation consists of **12 RESTful API services**. We decided to use these services for several reasons (1) they are publicly accessible to allow replication, including Canada Holidays and the Bills API collected from the website APIs.guru [10], (2) most of them are previously investigated in prior studies, including PetStore in [1], [4], [11], [12], Genome Nexus in [13], [14], [15], BingMap-Route in [16], [9], GitLab in [17], [16], [6], [18], [19] (3) ProShop [20], an in-house development, is a private

TABLE I
STATISTICS OF REST SERVICE DATASET

REST service	OP	2xx SC	4xx SC	Par	Dep Ops	Int Ops
ProShop	16	16	30	46	11	0
PetStore	19	15	20	61	10	0
Canada Holidays	6	6	2	9	2	0
Bills API	21	21	31	57	13	0
Genome Nexus	23	23	0	44	10	0
BingMap-Route	14	14	0	162	1	10
GitLab-Branch	9	9	0	88	7	1
GitLab-Commit	15	15	0	135	13	2
GitLab-Groups	17	17	0	152	15	0
GitLab-Issues	27	27	0	239	22	9
GitLab-Project	31	31	0	295	29	4
GitLab-Repository	10	10	0	101	8	2
Average	17.3	17	6.9	115.8	11.8	2.3

service with an OAS file that lies beyond the scope of GPT’s knowledge, ensuring fair comparison (4) they represent a diverse set of services from different domains with various status codes, operation dependencies, and parameter constraints.

Table I provides a summary of the services used in our experiments. *OP* is the number of operations defined in the OAS file for the service. *2xx SC* and *4xx SC* denote the counts of 2xx status codes and 4xx codes for all operations, respectively. *Par* is the total number of parameters, including those within the request bodies of operations. *DepOps* describes the number of operations with dependencies, and *IntOps* specifies the number of operations with at least one inter-parameter constraint.

V. PERFORMANCE ON TEST COVERAGE (RQ1)

A. Experimental Methodology

In the experiment, we applied both RTG (version 23.09) and our approach to all 12 services in the collected dataset (Table I). For each service, we analyzed the results of test case execution, with a specific focus on the final response status code of each generated test case.

To address RQ1, we formulated as follows:

2xx Coverage: It evaluates test coverage for successful responses by calculating the ratio of total 2xx response codes documented and triggered by at least one test case to the number of documented status codes within the 2xx range.

4xx Coverage: This assesses test coverage of failure responses by calculating the ratio of total 4xx response codes documented and triggered by at least one test case to the number of documented status codes within the 4xx range.

Overall Coverage: This measures the ratio of total actual response codes to the total documented status codes.

Average: We determine the average coverage for each measure by calculating the ratio of the total actual response codes across all services to the total number of status codes documented in the OAS files.

B. Experimental Results

Table II displays the coverage result. Our observations show an enhancement in the average coverage metrics over the state-of-the-art RTG. Specifically, we noted a 15.7% increase in

TABLE II
DOCUMENTED STATUS CODE COVERAGE (RQ1)

REST service	Overall coverage (%)		2xx coverage (%)		4xx coverage (%)	
	RTG	KAT	RTG	KAT	RTG	KAT
ProShop	67.4	87	62.5	100	70	80
Petstore	54	74.3	66.7	100	50	55
Canada Holidays	100	100	100	100	100	100
Bills API	71.2	82.7	66.7	81	74.2	83.9
Genome Nexus	78.3	100	78.3	100	-	-
BingMap-Route	28.6	78.6	28.6	78.6	-	-
GitLab-Branch	55.6	77.8	55.6	77.8	-	-
GitLab-Commit	20	20	20	20	-	-
GitLab-Groups	52.9	58.8	52.9	58.8	-	-
GitLab-Issues	40.7	63	40.7	63	-	-
GitLab-Project	67.7	74.2	67.7	74.2	-	-
GitLab-Repository	40	40	40	40	-	-
Average	59.2	74.9	56.4	74.5	67.5	75.9
Standard deviation	21.4	22.3	21.2	24.5	17.8	16.1

overall coverage, a 18.1% improvement in 2xx coverage, and a 8.4% increase in 4xx coverage, on average, over RTG.

We observed that for the BingMap Route service, which has the highest number of operations containing inter-parameter dependency constraints, KAT exhibits a high improvement of up to 50% in both overall coverage and 2xx coverage. Unlike RTG, which assigned values without considering inter-parameter dependency constraints, KAT, powered by GPT, effectively identifies and accommodates these constraints, generating valid test data that align with the detected constraints. In GitLab subsystems, particularly GitLab Issues, which features the highest number of operations containing inter-parameter dependency constraints, KAT has shown significant improvements in both overall coverage and 2xx coverage, achieving up to a 22.3% increase. In the services characterized by a high level of operation dependency, it attains a 100% 2xx coverage for ProShop, outperforming RTG, which achieves 62.5% coverage. This result validates our design intuition on capturing better dependencies, leading to better coverage.

Several cases correctly identify sequences to test an operation, but the responses of subsequent operations often prove inadequate to support requests for the next operation, especially when responses are empty or there are mismatches between the operation’s implementation and its specification. This issue is prominent in certain operations within GitLab’s services (GitLab Commit, GitLab Repository) or the Bills API, leading to incomplete testing sequences.

VI. TEST GENERATION EFFICIENCY (RQ2)

A. Experimental Methodology

To answer RQ2, we devised a measure to assess the efficiency of test case generation:

$$\text{Efficiency score}_R = \frac{\text{No. TCs actually covering SC in } R}{\text{No. TCs generated to cover SC in } R}$$

with R : 2xx or 4xx range.

No. TCs actually covering SC in R: the number of test cases that genuinely cover status codes (SC in Table I) in the

TABLE III
TEST CASE GENERATION EFFICIENCY (RQ2)

REST service	No. test cases generated to cover 2xx SC		No. test cases generated to cover 4xx SC		No. test cases actually covering 2xx SC		No. test cases actually covering 4xx SC		2xx score (%)		4xx score (%)	
	RTG	KAT	RTG	KAT	RTG	KAT	RTG	KAT	RTG	KAT	RTG	KAT
ProShop	711	19	569	190	10	16	21	24	1.4	84.2	3.7	12.6
Petstore	707	24	956	276	10	15	10	11	1.4	62.5	1	4
Canada Holidays	6	6	2	2	6	6	2	2	100	100	100	100
Bills API	1731	132	232	376	14	17	23	26	0.8	12.9	9.9	6.9
Genome Nexus	600	36	-	-	18	23	-	-	3	63.9	-	-
BingMap-Route	1413	35	-	-	4	11	-	-	0.3	31.4	-	-
GitLab-Branch	704	105	-	-	5	7	-	-	0.7	6.7	-	-
GitLab-Commit	4256	267	-	-	3	3	-	-	0.1	1.1	-	-
GitLab-Groups	3731	650	-	-	9	10	-	-	0.2	1.5	-	-
GitLab-Issues	5674	654	-	-	11	17	-	-	0.2	2.6	-	-
GitLab-Project	2008	372	-	-	21	23	-	-	1	6.2	-	-
GitLab-Repository	969	315	-	-	4	4	-	-	0.4	1.3	-	-
Average	1875.8	217.9	439.8	211	9.6	12.7	14	15.8	0.5	5.8	3.2	7.5

R range. In an operation, we consider its multiple test cases returning the same status code value as one.

No. TCs generated to cover SC in R: the number of test cases generated to encompass status codes (SC in Table I) in the R range. This variable calculates the minimum number of test cases required to cover SC in the R range, updating the count if new status codes in the same range are covered.

The higher coverage with the lower number of generated test cases implies a higher efficiency score.

Average: we determine average coverage by dividing the total number of test cases that actually cover status codes within the R range, across all services, by the total number of test cases generated to cover status codes within that range.

B. Experimental Results

Table III illustrates that the test suite generated by KAT comprises fewer instances where the expected coverage is not achieved compared to RTG. Notably, with the ProShop service, KAT successfully covers 84.2% of the 19 test cases designed for 2xx status codes, while RTG achieves successful coverage for 1.4% out of the 711 test cases it generates. Moreover, KAT shows coverage for 4xx status codes in 12.6% of the 190 generated test cases, in contrast to RTG's 3.7% success rate out of the 569 test cases it generates. In general, KAT generates significantly fewer test cases for each coverage type compared to RTG, yet it improves coverage for each service, except for Canada Holidays, where a basically valid value assigned to the required parameter can result in a successful response.

Overall, KAT enhances the efficiency in generating test cases, achieving a 5.3% improvement for 2xx status codes and a 4.3% improvement for 4xx status codes. These results affirm that KAT enhances the efficiency of test case generation, with the generated test cases considering operation dependencies and aligning request data with parameter constraints.

VII. FAILURE DETECTION (RQ3)

A. Experimental Methodology

Errors detection: the number of errors, which is the count of requests that result in server error status codes (5xx).

TABLE IV
FAILURE DETECTION (RQ3)

REST service	500 errors		Undocumented status codes		Mismatches	
	RTG	KAT	RTG	KAT	RTG	KAT
ProShop	7	6	0	0	21	14
Petstore	9	8	11	13	15	13
Canada Holidays	0	0	2	2	7	5
Bills API	0	1	0	0	26	26
Genome Nexus	0	0	15	14	14	16
BingMap-Route	4	5	31	32	6	6
GitLab-Branch	0	0	15	19	4	5
GitLab-Commit	1	0	19	21	3	0
GitLab-Groups	1	1	26	28	5	3
GitLab-Issues	1	1	37	48	8	3
GitLab-Project	0	1	50	53	8	0
GitLab-Repository	0	0	15	15	2	3
Total	23	23	221	245	119	94

Undocumented status codes detection: the number of response status codes that are absent from the OAS files.

Mismatches detection: we rely on trustworthy OAS files. Some previous tools neglected inter-parameter dependencies, leading to false-positive test cases for 2xx status codes. These false-positives occur when the tools listed them as failure cases to test 2xx status codes. Instead, KAT leverages these cases to improve the accuracy of testing 4xx status codes, resulting in more reliable coverage for 4xx status codes while allowing precise calculation of 2xx status code coverage.

B. Experimental Results

Table IV shows an equal number of errors detected by both RTG and KAT. The variation in error count between each service does not surpass one error.

In the realm of **undocumented** status code detection, KAT surpasses RTG, detecting 24 more status codes than RTG. Subsequent investigation underscores the prowess of our approach in identifying previously **undocumented** status codes, a proficiency that is significantly enhanced when operational dependencies are accurately detected. Consequently, our approach excels at detecting **undocumented** status codes, encom-

passing 304, 400, 422 and more. These findings underscore the distinct advantage of our approach in the detection of previously **undocumented** status codes.

Table IV reveals a notable difference in the average number of mismatches detected between RTG and our approach, KAT. RTG detects totally 119 mismatches, whereas KAT identifies 94 mismatches. The disparity in mismatch detection can be attributed to several factors. RTG sometimes fails to consider inter-parameter dependency constraints or overlooks fixed parameter values, resulting in the generation of false-positive test cases designed to cover 2xx status codes. Additionally, RTG may incorrectly detect crucial input parameters when subjected to mutation strategies, leading to the creation of false-positive test cases targeting 4xx status codes.

Regarding RQ3, RTG and KAT show similar capabilities in detecting errors. KAT adeptly detects status codes absent from the OAS file by considering both operational and inter-parameter dependencies. There is also a notable reduction in false-positive test cases from KAT.

VIII. THREATS TO VALIDITY

There are concerns that may affect the validity of our results.

Internal validity. One threat is the hallucination issue of GPT that results in inconsistent outputs when generating ODG, validation scripts, test cases, and test data. We alleviated this threat by using detailed and narrow instructions in prompts and by setting GPT’s temperature parameter to zero to control the model’s randomness. In addition, generated test cases were run against the working service as a method to validate and reduce the variation in GPT’s outputs.

Another threat is concerned with the lack of certainty in test oracles. By using the real-world services as black-box, we do not know whether the specification or the service implementation is accurate. It is possible that the specification is not up-to-date, or the service has errors. Due to this uncertainty, we reported mismatches in status code between the specification and implementation rather than actual errors found.

External validity. Although we evaluated the approaches using real-world applications, they may not represent the general population of API services. We mitigated this by choosing a diverse set of services, in terms of the application domain, number of operations and parameters, operation dependencies, business rules, and details of operation and parameter descriptions. They are publicly accessible, and many of them were evaluated in previous studies. Thus, the use of these services allows replication and cross-examination in future studies.

IX. RELATED WORK

Considering various surveys [13], [21], [22], [23], [24], [25], [26], previous approaches to RESTful APIs testing can be classified into two primary directions: black-box testing [1], [2], [4], [6], [7], [16], [27] and white-box testing [28], [29], [30], [31], [32], [33], [34], [35].

Resource dependencies. Viglianisi *et al.* introduced the use of ODG for modeling data dependencies among operations using OpenAPI specifications [1]. Liu *et al.* presented

the RESTful-service Property Graph (RPG) to provide detailed modeling of producer-consumer dependencies, including schema property equivalence relations. RPG can describe RESTful service behaviors and dynamically update itself with execution feedback [4]. Atlidakis *et al.* proposed RESTler to infer dependencies among request types declared in Swagger using a test-generation grammar [6]. Tree-based approaches, like linkage model trees [36] and API trees [19], were also considered. In contrast, we focus on constructing the ODG, leveraging natural language descriptions in specifications to guide LLMs for support and revision.

Inter-parameter dependency. Martin-Lopez *et al.* proposed Inter-parameter Dependency Language (IDL), a domain-specific language for analyzing dependencies among input parameters [37]. RESTest added inter-parameter dependencies to OpenAPI specifications using the IDL [37], implementing Constraint-Based Testing mode [2]. Wu *et al.* applied NLP for extracting inter-parameter constraints using pattern-based methods [16]. These approaches formalize inter-parameter dependencies, often requiring manual addition of IDL language or pre-defined NLP patterns, which is time-consuming and potentially error-prone. In contrast, Kim *et al.* used Reinforcement Learning to explore dependencies [14], Mirabella *et al.* trained a deep-learning model [38], while we leverage LLMs to extract inter-parameter dependencies from OAS files.

Natural language processing in API testing. Kim *et al.* introduced NLPtoREST, using NLP techniques to add supplementary OpenAPI rules from the human-readable specification [15]. Wanwarang *et al.* applied NLP to extract concepts associated with labels, used to query knowledge bases for input values [39]. ARTE explored various API elements, generating realistic test inputs by querying knowledge bases [5]. Liu *et al.* introduced RESTInfer, a two-phase approach inferring parameter constraints from RESTful API descriptions [40].

X. CONCLUSION

KAT is an AI-driven method for automated black-box testing of RESTful APIs. It uses a large language model and advanced prompting techniques to identify dependencies among operations, input parameters, and between operations and their parameters. KAT incorporates ODG construction in test script creation, detects inter-parameter dependencies, and considers parameter constraints to generate test data on target APIs.

We evaluated KAT using 12 real-world RESTful API services. The results show that it can automatically generate test cases and data to improve status code coverage, detect more **undocumented** status codes, and reduces more false positives in these services compared to RTG. It improves an overall status code coverage by 15.7% over RTG, increasing the coverage from 59.2% status codes by RTG to 74.9%.

ACKNOWLEDGMENTS

Vu Nguyen is partially supported by Vietnam’s NAFOSTED grant NCUD.02-2019.72. Tien N. Nguyen is supported in part by the US NSF grant CNS-2120386 and the NSA grant NCAE-C-002-2021.

REFERENCES

- [1] E. Viglianisi, M. Dallago, and M. Ceccato, “RestTestGen: automated black-box testing of restful APIs,” in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 142–152.
- [2] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, “REStest: Automated Black-Box Testing of RESTful Web APIs,” in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA ’21. Association for Computing Machinery, 2021.
- [3] A. Arcuri, “Restful api automated test case generation with evomaster,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 28, no. 1, pp. 1–37, 2019.
- [4] Y. Liu, Y. Li, G. Deng, Y. Liu, R. Wan, R. Wu, D. Ji, S. Xu, and M. Bao, “Morest: model-based restful api testing with execution feedback,” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1406–1417.
- [5] J. C. Alonso, A. Martin-Lopez, S. Segura, J. M. Garcia, and A. Ruiz-Cortés, “ARTE: Automated Generation of Realistic Test Inputs for Web APIs,” *IEEE Transactions on Software Engineering*, vol. 49, no. 1, pp. 348–363, 2022.
- [6] V. Atlidakis, P. Godefroid, and M. Polishchuk, “Restler: Stateful rest API fuzzing,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 748–758.
- [7] N. Laranjeiro, J. Agnelo, and J. Bernardino, “A black box tool for robustness testing of rest services,” *IEEE Access*, vol. 9, pp. 24 738–24 754, 2021.
- [8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [9] D. Corradini, A. Zampieri, M. Pasqua, and M. Ceccato, “RestTestGen: An Extensible Framework for Automated Black-box Testing of RESTful APIs,” in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022, pp. 504–508.
- [10] “Apis.guru,” 2024. [Online]. Available: <https://apis.guru/>
- [11] D. Corradini, A. Zampieri, M. Pasqua, and M. Ceccato, “Restats: A test coverage tool for RESTful APIs,” in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2021, pp. 594–598.
- [12] T. Vassiliou-Gioles, “A simple, lightweight framework for testing restful services with ttcn-3,” in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 2020, pp. 498–505.
- [13] M. Kim, Q. Xin, S. Sinha, and A. Orso, “Automated test generation for rest apis: No time to rest yet,” in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 289–301.
- [14] M. Kim, S. Sinha, and A. Orso, “Adaptive REST API Testing with Reinforcement Learning,” *arXiv preprint arXiv:2309.04583*, 2023.
- [15] M. Kim, D. Corradini, S. Sinha, A. Orso, M. Pasqua, R. Tzoref-Brill, and M. Ceccato, “Enhancing REST API Testing with NLP Techniques,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1232–1243.
- [16] H. Wu, L. Xu, X. Niu, and C. Nie, “Combinatorial testing of restful APIs,” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 426–437.
- [17] K. Yamamoto, “Efficient penetration of API sequences to test stateful RESTful services,” in *2021 IEEE International Conference on Web Services (ICWS)*. IEEE, 2021, pp. 734–740.
- [18] S. Karlsson, A. Čaušević, and D. Sundmark, “QuickREST: Property-based test generation of OpenAPI-described RESTful APIs,” in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 131–141.
- [19] J. Lin, T. Li, Y. Chen, G. Wei, J. Lin, S. Zhang, and H. Xu, “foREST: A Tree-based Approach for Fuzzing RESTful APIs,” *arXiv preprint arXiv:2203.02906*, 2022.
- [20] “Proshop API,” 2024. [Online]. Available: <https://anonymous.4open.science/r/proshop-api-B648>
- [21] A. Golmohammadi, M. Zhang, and A. Arcuri, “Testing RESTful APIs: A Survey,” *ACM Transactions on Software Engineering and Methodology*, 2022.
- [22] A. Ehsan, M. A. M. Abuhaliqa, C. Catal, and D. Mishra, “RESTful API testing methodologies: Rationale, challenges, and solution directions,” *Applied Sciences*, vol. 12, no. 9, p. 4369, 2022.
- [23] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, “Online testing of RESTful APIs: Promises and challenges,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 408–420.
- [24] A. Sharma, M. Revathi *et al.*, “Automated API testing,” in *2018 3rd International Conference on Inventive Computation Technologies (ICICT)*. IEEE, 2018, pp. 788–791.
- [25] B. Marculescu, M. Zhang, and A. Arcuri, “On the faults found in rest APIs by automated test generation,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 3, pp. 1–43, 2022.
- [26] A. Martin-Lopez, A. Arcuri, S. Segura, and A. Ruiz-Cortés, “Black-box and white-box test case generation for RESTful APIs: Enemies or allies?” in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2021, pp. 231–241.
- [27] D. Corradini, A. Zampieri, M. Pasqua, E. Viglianisi, M. Dallago, and M. Ceccato, “Automated black-box testing of nominal and error scenarios in RESTful APIs,” *Software Testing, Verification and Reliability*, vol. 32, no. 5, p. e1808, 2022.
- [28] O. Sahin and B. Akay, “A discrete dynamic artificial bee colony with hyper-scout for RESTful web service API test suite generation,” *Applied Soft Computing*, vol. 104, p. 107246, 2021.
- [29] A. Arcuri, “Automated black-and white-box testing of restful APIs with evomaster,” *IEEE Software*, vol. 38, no. 3, pp. 72–78, 2020.
- [30] A. Arcuri and J. P. Galeotti, “Enhancing search-based testing with testability transformations for existing APIs,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–34, 2021.
- [31] —, “Handling sql databases in automated system test generation,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–31, 2020.
- [32] M. Zhang and A. Arcuri, “Adaptive hypermutation for search-based system test generation: A study on rest APIs with EvoMaster,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–52, 2021.
- [33] —, “Enhancing resource-based test case generation for RESTful APIs with SQL handling,” in *International Symposium on Search Based Software Engineering*. Springer, 2021, pp. 103–117.
- [34] M. Zhang, B. Marculescu, and A. Arcuri, “Resource-based test case generation for RESTful web services,” in *Proceedings of the genetic and evolutionary computation conference*, 2019, pp. 1426–1434.
- [35] —, “Resource and dependency based test case generation for restful web services,” *Empirical Software Engineering*, vol. 26, no. 4, p. 76, 2021.
- [36] D. Stallenberg, M. Olsthoorn, and A. Panichella, “Improving test case generation for rest APIs through hierarchical clustering,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 117–128.
- [37] A. Martin-Lopez, “Automated analysis of inter-parameter dependencies in web APIs,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, 2020, pp. 140–142.
- [38] A. G. Mirabella, A. Martin-Lopez, S. Segura, L. Valencia-Cabrera, and A. Ruiz-Cortés, “Deep learning-based prediction of test input validity for restful APIs,” in *2021 IEEE/ACM Third International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*. IEEE, 2021, pp. 9–16.
- [39] T. Wanwarang, N. P. Borges Jr, L. Bettscheider, and A. Zeller, “Testing apps with real-world inputs,” in *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*, 2020, pp. 1–10.
- [40] Y. Liu, “RESTInfer: automated inferring parameter constraints from natural language RESTful API descriptions,” in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 1816–1818.